PRODUCT / ENGINEERING / OPERATIONS

MAYAK

A construction operations platform I designed, built and deployed for a construction company.

MAYAK brings project work, site records and procurement into one system. This case follows a purchase request from the site through finance and dispatch, then explains the data rules, access boundaries and recovery work behind it.

My scope

Requirements and workflow design; responsive interface; Next.js and TypeScript application; PostgreSQL routines and access policies; integrations; deployment; server migration; backups and ongoing production fixes.

Technical foundation

Next.js 16, React 19, TypeScript and PostgreSQL, with self-hosted Supabase for Auth, PostgREST and Storage. Docker and Nginx run the service on a VPS.

Read or watch

Repository and full technical notes

English walkthrough - 2 minutes 42 seconds

The walkthrough is an English visual reconstruction using invented sample data. Production records and application source are not included.

01 / PRODUCT CONTEXT

Keep the project, owner and next action clear.

The work crosses site teams, procurement, finance and management. Each participant needs a different view, but they must still be looking at the same project and request.

What the product covers

Project work: assigned teams, multiple task assignees, subtasks, checklists, comments, issues, dates, a Gantt view and acceptance.

Site records: photo reports, gallery uploads, worker and staff timesheets, and optional location evidence.

Materials and commercial work: movement-based inventory, purchase requests, invoice files, registration, documents, contracts, acts and payment records.

Two rules that make the records useful

A submitted task is not accepted work. The assignee submits it for review; the task author accepts or returns it. Management can resolve the review if the author has been deleted.

Material balances come from movements. Receipts, write-offs and transfers explain how the current balance was reached. Finance approval does not automatically create a material receipt: these remain separate modules.

Constraints

Phone use, weak site connectivity, a self-hosted stack and an operating company shaped the work. I chose incremental changes over replacing the running product in one release.

02 / A COMPLETE WORKFLOW

One purchase request, several owners.

1. Request and approval

A site manager requests plywood and accessories as separate material items, with a required date, priority and approver. Approval hands the request to procurement.

2. Supplier alternatives

Procurement attaches invoice files to each item. Groups preserve their title and comment. Every item needs an invoice before the request goes to finance.

3. File-level choice

Finance selects the actual files within those groups. Each material item must have a selected file. Alternatives remain in history, marked as not selected.

4. Two completion states

Procurement completes the order work. Dispatch independently registers the selected files. A linked register entry stays visible after registration; the request is not duplicated.

5. Corrections and shorter routes

Permitted actors can return work to an earlier stage or recall their own handoff. Direct invoices go to finance and dispatch without creating a supply completion step.

03 / DATA INTEGRITY

What the finance approval actually guarantees.

The finance decision updates several related records. I put that operation in a PostgreSQL routine rather than letting a sequence of interface requests partially complete it.

Inside the database transaction

Check the actor and scope. Require finance duty, an active company, the matching company and the finance-review stage.

Lock and validate. Lock the request row. Reject files from another request, empty selections or a material item with no chosen invoice.

Write the result together. Save per-file decisions and group summaries, change the request status, create or reset the registry entry, and append the event.

A separate guard for stale screens

Ordinary request transitions qualify their update with the current status. If another person has moved the request, the stale action must not overwrite the new state. These actions are not all the same transaction as their event writes; the stronger atomic claim applies to the finance routine.

Why the model matters

An invoice group is not a file. An approved request is not a registered invoice. Keeping those concepts separate prevents a single convenient status flag from hiding unfinished work.

04 / ARCHITECTURE AND ACCESS

Enforce the boundary beyond the interface.

Browser / PWA → Nginx over HTTPS → Next.js → the self-hosted Supabase gateway → Auth, PostgREST and Storage → PostgreSQL.

User-scoped paths

Ordinary server actions use the user session to access the data API. PostgreSQL row-level security constrains company and project visibility. Interface controls and server checks help explain permissions, but do not replace the database boundary.

Rank is not a department duty

Crew, engineer, site manager and management describe managerial authority. Supply, finance and dispatcher describe responsibilities. Duties can be combined without raising rank.

Privileged exceptions are explicit

Some administrative paths and database routines use elevated access. They must check the actor, company and permitted operation themselves. I do not claim that RLS alone protects every server operation.

Invoice downloads

The file handler authenticates the session and checks permitted file metadata before creating a 60-second storage URL. The server streams the response with its original filename, private/no-store caching and MIME controls. It does not expose a permanent public invoice URL.

05 / FIELD USE AND INTEGRATIONS

Support the handoff without redefining the state.

Uploading from a phone

Decodable images are resized to a maximum side of 1600 pixels and encoded as JPEG at quality 0.82. Already-small JPEGs are kept. If conversion fails, the original is returned instead of silently discarding the upload.

Connecting MAX

The site issues a random connection code; only its hash is stored. Redemption checks expiry and use, links the messenger identity, and consumes the token. A message links back to MAYAK, where the normal session and permissions still apply.

Delivery remains best-effort

Browser push and MAX are notification channels. A failed message must not become the source of truth for whether a request was approved. There is no durable notification outbox or read guarantee in this implementation.

Offline is a limited promise

The PWA provides an offline fallback page. It does not provide offline task editing or a private database that synchronizes later. I keep that limit explicit rather than presenting installation as full offline support.

06 / DELIVERY AND VERIFICATION

Operate it, recover it, improve it.

Migration and recovery

I moved the application from its original server to a VPS. The transfer included database roles and grants, file bytes and attributes, environment configuration, HTTPS and a controlled final handoff.

Versioned encrypted snapshots now include the database, roles, Storage and configuration, with an offsite repository. I restored an actual cloud snapshot into a separate directory and test database, checking file attributes and proxy configuration too.

What I checked

For this case study, I reran 12 focused tests covering MAX codes, the direct-invoice working-list regression and activity-report filters/export. All passed. Database regression scripts and a backup integration test also exist; this is not full browser end-to-end coverage.

Limits and result

One VPS remains a failure domain. Live files and the database are not one atomic cross-system snapshot. Notifications are best-effort. I have not measured an attributable ROI or a percentage time saving.

The delivered result is the running product and its implemented workflows, access model, integrations, migration and recovery procedure. My work spans requirements, design, application and database engineering, and ongoing operation.

Full case study, diagrams, walkthrough and technical notes